

CloudAsset Manager

Manual de Implementación: Sistema Automatizado de Gestión de
Activos en la Nube con AWS

Autor: Juan Carlos Lopez de Cardenas Avelar

Estudiante de Ingeniería de Software

Fecha: Mayo 2026

Contenido

Capítulo 1: Introducción al Proyecto	4
1.1 Propósito del Proyecto	4
1.2 Implicaciones en el Entorno Laboral Real	4
1.3 Glosario de Términos Conceptuales.....	5
Capítulo 2: Stack Tecnológico y Recursos de la Capa Gratuita.....	5
2.1 Descripción de Servicios y Conceptos Técnicos	5
2.2 Cuadro Comparativo de Recursos y Límites de la Capa Gratuita	6
Capítulo 3: Construcción de la Infraestructura Paso a Paso.....	7
3.1 Fase 1: Gestión de Identidad y Accesos (IAM)	7
3.1.1 Creación del Rol de Ejecución para AWS Lambda	7
3.1.2 Creación del Rol de Instancia para Amazon EC2	8
3.2 Fase 2: Segmentación y Aislamiento de Red (VPC)	9
3.2.1 Creación de la Red Virtual (VPC and more)	9
3.2.2 Configuración de los Grupos de Seguridad (Security Groups)	10
3.3 Fase 3: Aprovisionamiento de Almacenamiento (Amazon S3)	12
3.3.1 Creación y Configuración del Bucket.....	12
3.4 Fase 4: Implementación de la Base de Datos Relacional (Amazon RDS)	14
3.4.1 Configuración del Motor y Plantilla.....	14
3.4.2 Credenciales y Especificaciones de Cómputo.....	15
3.4.3 Conectividad y Aislamiento de Red	15
3.4.4 Inicialización de la Base de Datos	16
3.4.5 Ejecución del Lanzamiento	16
3.5 Fase 5: Automatización Serverless y Lógica de Negocio (AWS Lambda)	17
3.5.1 Definición de la Función y Entorno de Ejecución	17
3.5.2 Integración con la Red Privada (VPC)	17
3.5.3 Configuración del Disparador (S3 Trigger)	18
3.5.4 Ajustes Avanzados y Enrutamiento (Configuración Crítica)	18
3.5.5 Implementación del Código Fuente	18
3.6 Fase 6: Despliegue del Servidor Web y Visualización (Amazon EC2)	20
3.6.1 Aprovisionamiento y Configuración de la Instancia	20
3.6.2 Conexión Segura mediante AWS Systems Manager	21
3.6.3 Preparación del Entorno Operativo (Node.js)	21
3.6.4 Desarrollo del Servidor Web (server.js).....	22
3.6.5 Ejecución y Verificación de Acceso.....	24

3.7 Fase 7: Observabilidad, Monitoreo y Alertas (Amazon CloudWatch)	25
3.7.1 Creación del Tablero de Monitoreo (Dashboard)	25
3.7.2 Configuración del Sistema de Alertas Automáticas	26
Capítulo 4: Pruebas de Integración y Resolución de Problemas	27
4.1 Protocolo de Pruebas de Funcionamiento	27
4.2 Resolución de Errores Comunes (Troubleshooting)	28
Conclusión del Manual	29
Referencias y Bibliografía Técnica	30

Capítulo 1: Introducción al Proyecto

1.1 Propósito del Proyecto

En la era digital contemporánea, las organizaciones generan, almacenan y procesan cantidades masivas de información en forma de archivos multimedia, documentos y conjuntos de datos. La gestión manual de estos recursos resulta ineficiente, propensa a errores y difícil de escalar.

El presente proyecto, denominado **CloudAsset Manager**, propone la construcción de un sistema integral que automatiza el ciclo de vida de los datos desde el momento en que ingresan a la infraestructura de una organización. El objetivo principal es desarrollar una plataforma donde la simple acción de cargar un archivo desencadene una serie de procesos automáticos que analicen el archivo, extraigan sus características fundamentales (metadatos) y registren esta información en una base de datos segura y privada. Finalmente, el sistema expone este inventario en tiempo real a través de un portal web accesible para los administradores o usuarios finales.

A lo largo de este manual, se detallará paso a paso la construcción de esta arquitectura, priorizando las mejores prácticas de seguridad, el aislamiento de redes y la optimización de recursos.

1.2 Implicaciones en el Entorno Laboral Real

En el mercado laboral actual, las empresas buscan alejarse de los servidores tradicionales que requieren mantenimiento constante y generan costos operativos continuos, incluso cuando no están siendo utilizados. Este proyecto aborda problemas reales de la industria mediante la implementación de patrones de diseño modernos que las grandes corporaciones utilizan a diario.

Las implicaciones directas de esta arquitectura en el mundo real incluyen:

- **Eficiencia Operativa:** Al eliminar la necesidad de que un operador humano registre manualmente cada archivo que ingresa al sistema, las empresas pueden redirigir el talento hacia tareas de mayor valor analítico.
- **Escalabilidad Dinámica:** El modelo construido en este proyecto permite que el sistema procese desde un archivo al día hasta miles de archivos por minuto sin requerir rediseños en la infraestructura. Es el mismo principio que utilizan las plataformas de *streaming* de video al procesar contenido subido por millones de usuarios.
- **Reducción de Costos:** Al adoptar un enfoque donde los procesos lógicos solo se ejecutan (y se facturan) en el momento exacto en que llega un archivo, se elimina el concepto de "tiempo de inactividad pagado".
- **Seguridad y Cumplimiento:** La segmentación del sistema garantiza que la información estructurada permanezca completamente aislada de la red pública, un requisito fundamental para empresas en sectores financieros o de salud que manejan datos sensibles.

1.3 Glosario de Términos Conceptuales

Para asegurar la comprensión total de los principios aplicados en este manual, a continuación se definen los conceptos arquitectónicos clave:

- **Activo Digital (Digital Asset):** Cualquier elemento de contenido estructurado en un formato digital (imágenes, documentos, videos) que posee valor para la organización.
- **Metadatos (Metadata):** Datos que describen otros datos. En el contexto de este proyecto, se refiere a las propiedades de un archivo, como su nombre exacto, su tamaño en bytes y la fecha precisa de su creación o carga.
- **Arquitectura Orientada a Eventos (Event-Driven Architecture):** Un patrón de diseño de software en el que un programa o servicio no se ejecuta de forma continua, sino que permanece inactivo hasta que es "despertado" o desencadenado por una acción específica (un evento), como la llegada de un nuevo archivo a un repositorio.
- **Computación sin Servidor (Serverless Computing):** Un modelo de ejecución en la nube en el que el proveedor asigna dinámicamente los recursos de la máquina, permitiendo a los desarrolladores ejecutar código sin tener que aprovisionar, configurar ni administrar servidores físicos o virtuales de manera manual.
- **Segmentación de Red (Network Segmentation):** La práctica de dividir una red informática en subredes más pequeñas y aisladas. Esto mejora la seguridad y el rendimiento al controlar estrictamente qué partes del sistema pueden comunicarse con el exterior y cuáles permanecen ocultas.

Capítulo 2: Stack Tecnológico y Recursos de la Capa Gratuita

Para la consolidación de este proyecto, se ha seleccionado un conjunto de servicios de **Amazon Web Services (AWS)** que, combinados, forman una arquitectura robusta, escalable y segura. Es imperativo destacar que toda la configuración detallada en este manual ha sido diseñada para operar bajo el **AWS Free Tier (Capa Gratuita)**. Esto permite a profesionales y estudiantes desarrollar infraestructuras de nivel empresarial sin incurrir en costos operativos, siempre que se respeten los límites de uso establecidos por el proveedor.

A continuación, se describen los servicios que integran nuestro *Tech Stack*, definiendo sus funciones principales y los componentes técnicos que los hacen operativos.

2.1 Descripción de Servicios y Conceptos Técnicos

- **AWS Identity and Access Management (IAM):** Es el servicio que administra la seguridad y el acceso a los recursos. Funciona mediante la creación de **Roles**, que son identidades con permisos específicos que un servicio asume para realizar tareas en nombre del usuario, y **Políticas**, que son documentos de configuración que definen exactamente qué acciones están permitidas o prohibidas.

- **Amazon Virtual Private Cloud (VPC):** Es nuestra sección lógicamente aislada de la nube de AWS donde desplegamos recursos en una red virtual definida por nosotros. Se compone de **Subredes** (segmentos de la red para dividir el tráfico), **Tablas de Enrutamiento** (reglas que deciden hacia dónde se dirige el tráfico) y **Grupos de Seguridad**, que actúan como un *firewall* virtual para controlar el tráfico entrante y saliente.
- **Amazon Simple Storage Service (Amazon S3):** Servicio de almacenamiento de objetos diseñado para guardar y recuperar cualquier cantidad de datos. Utiliza **Buckets**, que son contenedores fundamentales para el almacenamiento, y trata a cada archivo como un **Objeto**.
- **Amazon Relational Database Service (Amazon RDS):** Facilita la configuración, operación y escalabilidad de una base de datos relacional. En este proyecto, utilizamos el motor **MySQL**, que organiza la información en tablas vinculadas por claves comunes, garantizando la integridad de los metadatos.
- **AWS Lambda:** Servicio de computación que permite ejecutar código sin aprovisionar ni administrar servidores. Se basa en un **Entorno de Ejecución (Runtime)**, en nuestro caso Node.js, y se activa mediante un **Disparador (Trigger)**, que es el evento (como la carga de un archivo en S3) que inicia la lógica del código.
- **Amazon Elastic Compute Cloud (Amazon EC2):** Proporciona capacidad de cómputo segura y redimensionable a través de servidores virtuales. Estos se lanzan a partir de una **AMI (Amazon Machine Image)**, que es una plantilla preconfigurada con el sistema operativo (Linux) y el software necesario para hospedar nuestra aplicación web.
- **Amazon CloudWatch:** Es el sistema de monitoreo y observabilidad. Recopila **Métricas** (datos numéricos sobre el rendimiento) y **Logs** (registros de texto de eventos del sistema) para asegurar que la infraestructura funcione según lo previsto.

2.2 Cuadro Comparativo de Recursos y Límites de la Capa Gratuita

Para garantizar que el proyecto se mantenga con un costo de \$0.00 USD, se presenta la siguiente tabla con las funciones específicas de cada servicio dentro de nuestra arquitectura y los límites mensuales que no deben excederse.

<i>Servicio</i>	<i>Función en el Proyecto</i>	<i>Límite de la Capa Gratuita (Mensual)</i>
IAM	Gestión de permisos seguros para la intercomunicación de servicios.	Siempre gratuito (sin límites operativos).
VPC	Segmentación de red con subredes públicas y privadas aisladas.	Configuración de red estándar gratuita.
S3	Repositorio central para la carga de activos digitales (archivos).	5 GB de almacenamiento estándar y 2,000 solicitudes de carga.

RDS	Almacenamiento estructurado de metadatos en un motor MySQL.	750 horas de instancia tipo db.t2.micro o db.t3.micro.
EC2	Servidor web para la visualización del inventario en tiempo real.	750 horas de instancia tipo t2.micro o t3.micro.
Lambda	Procesamiento automático de archivos mediante lógica <i>Serverless</i> .	1 millón de solicitudes y 400,000 GB-segundos de cómputo.
CloudWatch	Monitoreo de actividad, generación de logs y alertas preventivas.	10 métricas detalladas y 5 GB de registros de logs.

Este ecosistema tecnológico asegura que el **CloudAsset Manager** no solo sea una solución funcional, sino también una implementación optimizada bajo estándares de eficiencia y ahorro de costos en la nube.

Capítulo 3: Construcción de la Infraestructura Paso a Paso

3.1 Fase 1: Gestión de Identidad y Accesos (IAM)

El primer pilar de cualquier arquitectura en la nube es la seguridad. En Amazon Web Services, esto se gestiona a través de **IAM (Identity and Access Management)**.

Para este proyecto, aplicaremos el **Principio de Menor Privilegio (Principle of Least Privilege)**, una práctica de seguridad informática que consiste en otorgar a un usuario, programa o proceso únicamente los permisos estrictamente necesarios para realizar su función, ni uno más.

En lugar de utilizar credenciales estáticas (como nombres de usuario y contraseñas incrustadas en el código, lo cual representa un riesgo crítico de seguridad), crearemos **Roles de IAM**. Un rol es una identidad temporal que un servicio de AWS puede asumir para comunicarse de forma segura con otro servicio.

A continuación, configuraremos las dos identidades fundamentales para nuestra arquitectura: el rol para el procesamiento automático (Lambda) y el rol para el servidor web (EC2).

3.1.1 Creación del Rol de Ejecución para AWS Lambda

Este rol autorizará a nuestra función sin servidor a leer el archivo recién cargado en el repositorio, registrar sus actividades y conectarse a la red privada donde residirá la base de datos.

1. **Acceso al Servicio:** Inicie sesión en la Consola de Administración de AWS. En la barra de búsqueda superior, escriba **IAM** y seleccione el servicio correspondiente.
2. **Inicio de Creación:** En el panel de navegación izquierdo, haga clic en la sección **Roles** y posteriormente presione el botón **Create role** (Crear rol).

3. Selección de la Entidad de Confianza (Trust Entity):

- En el tipo de entidad de confianza, seleccione **AWS service** (Servicio de AWS).
- En el apartado de caso de uso (Use case), seleccione **Lambda** en el menú desplegable.
- Haga clic en **Next** (Siguiendo).

4. Asignación de Políticas de Permisos: Una **Política (Policy)** es un documento que define explícitamente los permisos. Utilice la barra de búsqueda para localizar y marcar las casillas de las siguientes tres políticas administradas:

- **AWSLambdaBasicExecutionRole**: Permite a la función enviar registros de actividad y errores (logs) al servicio de monitoreo CloudWatch.
- **AmazonS3ReadOnlyAccess**: Otorga permiso de solo lectura para extraer información de los archivos almacenados en los buckets de S3.
- **AWSLambdaVPCAccessExecutionRole**: **Vital para la arquitectura**. Permite a la función crear interfaces de red elásticas para conectarse a los recursos alojados dentro de una red privada (VPC), como nuestra base de datos relacional.
- Tras seleccionar las tres políticas, haga clic en **Next**.

5. Revisión y Creación:

- En el campo **Role name** (Nombre del rol), asigne un identificador estandarizado y descriptivo: **CloudAsset-Lambda-Role**.
- Verifique en el resumen que las tres políticas estén adjuntas.
- Haga clic en el botón **Create role**.

3.1.2 Creación del Rol de Instancia para Amazon EC2

Este segundo rol será asignado al servidor virtual que alojará la página web. Su propósito principal es permitir la gestión segura del servidor sin necesidad de abrir puertos de conexión pública, maximizando la seguridad de la infraestructura.

1. Inicio de Creación: Regrese a la pantalla principal de la sección **Roles** y haga clic nuevamente en **Create role**.

2. Selección de la Entidad de Confianza:

- Tipo de entidad: **AWS service**.
- Caso de uso: Seleccione **EC2**.
- Haga clic en **Next**.

3. **Asignación de Políticas de Permisos:** Utilice la barra de búsqueda para localizar y seleccionar las siguientes políticas:

- AmazonS3ReadOnlyAccess: Otorga a la aplicación web la capacidad de listar y visualizar recursos del almacenamiento S3 si la escalabilidad del proyecto lo requiere en el futuro.
- AmazonSSMManagedInstanceCore: **Configuración avanzada de seguridad.** Esta política habilita el uso de AWS Systems Manager (SSM) Session Manager. Permite a los administradores abrir una terminal segura directamente desde el navegador web para configurar el servidor, eliminando por completo la necesidad de abrir el puerto SSH (Puerto 22) al internet público.
- Haga clic en **Next**.

4. **Revisión y Creación:**

- En el campo **Role name**, asigne el identificador: CloudAsset-EC2-Role.
- Haga clic en el botón **Create role**.

Validación de la Fase 1: Al finalizar este paso, el sistema cuenta con dos identidades operativas (CloudAsset-Lambda-Role y CloudAsset-EC2-Role) listas para ser asignadas a los componentes de procesamiento y visualización en las siguientes fases del despliegue.

3.2 Fase 2: Segmentación y Aislamiento de Red (VPC)

El núcleo de la seguridad en la nube no solo reside en quién tiene acceso (IAM), sino en **dónde** residen los recursos. A través de **Amazon Virtual Private Cloud (VPC)**, diseñaremos una topología de red de grado empresarial.

El diseño se basa en el principio de defensa en profundidad: los recursos que deben ser accedidos por los usuarios (como el servidor web) se ubicarán en **Subredes Públicas** (con acceso a internet), mientras que los recursos críticos (como la base de datos) permanecerán ocultos en **Subredes Privadas** (sin acceso directo desde internet).

A continuación, se describen los pasos exactos para construir esta red utilizando la herramienta de creación visual de AWS, aplicando configuraciones críticas para mantener el entorno libre de costos.

3.2.1 Creación de la Red Virtual (VPC and more)

Esta configuración construirá automáticamente la estructura base de la red, incluyendo las tablas de enrutamiento y las puertas de enlace necesarias.

1. **Acceso al Servicio:** En la consola de AWS, busque y seleccione el servicio **VPC**.
2. **Inicio del Asistente:** En el panel superior derecho, haga clic en el botón **Create VPC** (Crear VPC).

3. **Configuración de Arquitectura (Resources to create):** * Seleccione la opción **VPC and more** (VPC y más). Esto desplegará un mapa visual interactivo de la infraestructura.
4. **Asignación de Nombres y Direccionamiento IP:**
 - **Name tag auto-generation:** Ingrese el prefijo CloudAsset. AWS lo utilizará para nombrar todos los componentes de la red de forma estructurada.
 - **IPv4 CIDR block:** Mantenga el valor predeterminado 10.0.0.0/16. Esto proporciona un espacio de más de 65,000 direcciones IP privadas, lo cual es el estándar para arquitecturas escalables.
5. **Configuración de Subredes (Subnets):**
 - **Number of Availability Zones (AZs):** Seleccione **2**. *(Nota técnica: Amazon RDS requiere por diseño un mínimo de dos zonas de disponibilidad para garantizar la tolerancia a fallos, incluso en despliegues de capa gratuita).*
 - **Number of public subnets:** Seleccione **2**.
 - **Number of private subnets:** Seleccione **2**.
6. **Gestión de Tráfico y Costos (¡Configuración Crítica!):**
 - **NAT gateways (\$):** Seleccione explícitamente **None** (Ninguno).
 - *Definición Técnica:* Un NAT Gateway permite a los recursos en subredes privadas descargar actualizaciones de internet. Sin embargo, es un servicio de pago por hora y retiene direcciones IP elásticas que generan costos adicionales. Al seleccionar "None", garantizamos que la arquitectura se mantenga en **\$0.00 USD**.
 - **VPC endpoints:** Seleccione **S3 Gateway**.
 - *Definición Técnica:* Al no tener un NAT Gateway, nuestra subred privada queda incomunicada del exterior. El "S3 Gateway Endpoint" crea un túnel privado y gratuito dentro de la red de AWS que permitirá a nuestra función Lambda (en la subred privada) depositar o leer datos del almacenamiento S3 sin necesidad de salir a internet.
7. **Resolución de Nombres:** Asegúrese de que las opciones **Enable DNS hostnames** y **Enable DNS resolution** estén habilitadas.
8. Haga clic en **Create VPC**. Espere a que el flujo de trabajo finalice la creación de todos los recursos y presione "View VPC".

3.2.2 Configuración de los Grupos de Seguridad (Security Groups)

Un **Grupo de Seguridad** actúa como un firewall virtual a nivel de instancia que controla el tráfico entrante (Inbound) y saliente (Outbound). Para garantizar una conexión sin errores durante la fase de desarrollo, configuraremos tres grupos distintos con reglas precisas.

En el menú lateral izquierdo de la consola de VPC, navegue hasta la sección **Security** y seleccione **Security Groups**.

A. Grupo de Seguridad para el Servidor Web (EC2): CloudAsset-EC2-SG

Este firewall protegerá el servidor web, permitiendo el tráfico de los usuarios y habilitando una vía de administración remota para el desarrollador.

1. Haga clic en **Create security group**.
2. **Nombre:** CloudAsset-EC2-SG | **Descripción:** Permite trafico HTTP y SSH entrante.
3. **VPC:** Asegúrese de seleccionar la red recién creada (CloudAsset-vpc).
4. **Reglas de Entrada (Inbound rules):**
 - Regla 1 (Usuarios): Type **HTTP** | Port **80** | Source **Anywhere-IPv4** (0.0.0.0/0).
 - Regla 2 (Administración): Type **SSH** | Port **22** | Source **Anywhere-IPv4** (0.0.0.0/0).
 - *Corrección Preventiva:* La apertura del puerto 22 es indispensable para poder utilizar la herramienta "EC2 Instance Connect" desde el navegador. Sin esta regla, los intentos de acceder a la terminal del servidor resultarán en un error de conexión (Timeout).
5. Haga clic en **Create security group**.

B. Grupo de Seguridad para la Función Serverless (Lambda): CloudAsset-Lambda-SG

Este firewall se asignará al código que procesa los archivos.

1. Haga clic en **Create security group**.
2. **Nombre:** CloudAsset-Lambda-SG | **Descripción:** Grupo de seguridad para el procesador Lambda.
3. **VPC:** Seleccione CloudAsset-vpc.
4. **Reglas de Entrada:** No agregue ninguna regla. Las funciones Lambda inician el tráfico hacia la base de datos (Outbound), pero no reciben peticiones directas de red en esta arquitectura.
5. Haga clic en **Create security group**.

C. Grupo de Seguridad para la Base de Datos (RDS): CloudAsset-RDS-SG

Este es el firewall más restrictivo. Blindará la base de datos para que ignore cualquier petición proveniente de internet, aceptando únicamente conexiones de nuestras propias aplicaciones.

1. Haga clic en **Create security group**.

2. **Nombre:** CloudAsset-RDS-SG | **Descripción:** Permite trafico MySQL interno desde EC2 y Lambda.
3. **VPC:** Seleccione CloudAsset-vpc.
4. **Reglas de Entrada (Inbound rules):**
 - Regla 1 (Conexión EC2): Type **MySQL/Aurora** | Port **3306** | Source: Busque y seleccione el grupo de seguridad CloudAsset-EC2-SG.
 - Regla 2 (Conexión Lambda): Type **MySQL/Aurora** | Port **3306** | Source: Busque y seleccione el grupo de seguridad CloudAsset-Lambda-SG.
 - *Nota Técnica:* En lugar de utilizar direcciones IP, encadenar los Grupos de Seguridad garantiza que, aunque las IPs internas de los servidores cambien, la confianza entre ellos se mantiene intacta.
5. Haga clic en **Create security group**.

Validación de la Fase 2: La infraestructura de red base está completa. Contamos con subredes estratégicamente posicionadas, protección contra cobros accidentales al omitir el NAT Gateway, y un esquema de firewalls interconectados que asegura que todos los componentes podrán comunicarse entre sí en los pasos posteriores sin generar errores de rechazo de conexión.

3.3 Fase 3: Aprovisionamiento de Almacenamiento (Amazon S3)

Una vez que la red (VPC) y los permisos (IAM) están establecidos, procederemos a desplegar el primer servicio de infraestructura física virtualizada: **Amazon Simple Storage Service (S3)**.

En el contexto de nuestra arquitectura *Event-Driven* (Orientada a Eventos), S3 actuará como el "detonador" o punto de entrada. Es el repositorio donde los sistemas externos o usuarios depositarán los activos digitales. S3 es ideal para este propósito debido a su escalabilidad infinita y su capacidad inherente de enviar notificaciones (eventos) a otros servicios de AWS en el instante en que un archivo (objeto) es modificado o cargado.

El siguiente procedimiento garantizará la creación de un repositorio (Bucket) que permanezca completamente privado, protegiendo los activos digitales contra accesos no autorizados desde el exterior, mientras permite el acceso interno seguro a través del *S3 Gateway Endpoint* que configuramos en la Fase 2.

3.3.1 Creación y Configuración del Bucket

1. **Acceso al Servicio:** En la consola de administración de AWS, busque y seleccione el servicio **S3**.
2. **Inicio del Proceso:** En el panel de control principal, presione el botón **Create bucket** (Crear bucket).
3. **Configuración General:**

- **Bucket name (Nombre del bucket):** Ingrese un identificador único que siga convenciones de nomenclatura estandarizadas.
- *Regla Técnica:* El nombre debe estar en minúsculas, sin espacios y, lo más crítico, **debe ser globalmente único** en todo el ecosistema de AWS. Se sugiere utilizar el formato: cloudasset-inventario-[sus-iniciales]-2026.
- **AWS Region:** Seleccione **exactamente la misma región** donde implementó la VPC en el paso 2.2. (Ej. *us-east-2 Ohio*).
- *Corrección Preventiva:* La disparidad de regiones entre S3 y otros servicios como EC2 o Lambda puede resultar en costos imprevistos por "Transferencia de Datos entre Regiones" y aumento de latencia.

4. Controles de Propiedad (Object Ownership):

- Mantenga seleccionada la opción **ACLs disabled (recommended)**.
- *Justificación:* En arquitecturas en la nube modernas, el control de acceso se delega íntegramente a las Políticas de IAM (creadas en la Fase 1), haciendo obsoletas y riesgosas las Listas de Control de Acceso (ACLs) individuales por objeto.

5. Configuración de Seguridad Pública (Block Public Access settings):

- Verifique que la opción **Block all public access** (Bloquear todo el acceso público) se encuentre activada.
- *Justificación:* Nuestro diseño dicta que ningún usuario final accederá directamente al bucket. Los archivos serán consultados por la infraestructura interna a través de los roles IAM. Mantener el bucket bloqueado previene la exposición accidental de datos y posibles ataques de denegación de cartera (agotamiento intencional de la capa gratuita).

6. Configuración de Versionado (Bucket Versioning):

- Asegúrese de que el estado sea **Disable** (Desactivado).
- *Nota de Costos:* Habilitar el versionado implica que cada vez que se sobrescriba un archivo, AWS almacenará y facturará por el tamaño de ambas versiones simultáneamente, lo cual es innecesario para los propósitos de este entorno de prueba.

7. Cifrado en Reposo (Default encryption):

- AWS encripta los datos por defecto para cumplir con normativas de seguridad. Confirme que la opción **Server-side encryption with Amazon S3 managed keys (SSE-S3)** esté seleccionada y que el campo **Bucket Key** permanezca en **Enable**.

8. Finalización: Desplácese hasta el final del formulario y haga clic en **Create bucket**.

Validación de la Fase 3: El repositorio de datos crudos se encuentra aprovisionado, cifrado y aislado del acceso público, listo para comenzar a recibir eventos y comunicarse de forma privada con el resto de la topología en la nube.

3.4 Fase 4: Implementación de la Base de Datos Relacional (Amazon RDS)

Con el repositorio de archivos preparado, el siguiente componente es el motor de almacenamiento estructurado. **Amazon Relational Database Service (RDS)** es un servicio administrado que simplifica la configuración, operación y escalabilidad de una base de datos relacional en la nube, liberando al administrador de tareas de mantenimiento del sistema operativo.

Para este proyecto, utilizaremos el motor **MySQL**. Su función será almacenar los metadatos exactos de cada archivo (nombre, tamaño y fecha de registro) que nuestra lógica sin servidor (Lambda) extraerá.

A continuación, se detalla el procedimiento de creación, haciendo especial énfasis en la configuración de red para garantizar el aislamiento (colocando la base de datos en el "sótano" privado de nuestra VPC) y en la desactivación de características automatizadas que podrían generar facturación no deseada.

3.4.1 Configuración del Motor y Plantilla

1. **Acceso al Servicio:** En la barra de búsqueda superior de la consola de AWS, escriba **RDS** y seleccione el servicio.
2. **Inicio del Aprovisionamiento:** En el panel de control principal, haga clic en el botón naranja **Create database** (Crear base de datos).
3. **Método de Creación (Creation method):** Seleccione **Standard create** (Creación estándar).
 - *Justificación:* A diferencia de la creación "fácil", la estándar nos permite inyectar la base de datos específicamente dentro de nuestra VPC y asignar nuestro propio firewall (Grupo de Seguridad).
4. **Opciones del Motor (Engine options):** Seleccione **MySQL**.
5. **Plantilla de Despliegue (Templates - ¡Protección Anti-Cobros!):** Seleccione estrictamente la opción **Free tier** (Capa gratuita).
 - *Justificación Crítica:* Seleccionar esta plantilla desactiva automáticamente características empresariales costosas, como el despliegue Multi-AZ (Múltiples zonas de disponibilidad, que duplica los servidores para alta disponibilidad) y los clústeres de IOPS aprovisionadas, asegurando que la arquitectura base se mantenga en \$0.00 USD.

3.4.2 Credenciales y Especificaciones de Cómputo

1. Configuración de la Instancia:

- **DB instance identifier (Identificador de la instancia):** Escriba cloudasset-db. Este es el nombre del hardware virtual, no de la base de datos interna.
- **Master username (Usuario maestro):** Ingrese admin.
- **Master password (Contraseña maestra):** Defina una contraseña segura alfanumérica y guárdela de forma segura. *Nota: Esta contraseña será requerida posteriormente en los códigos fuente de Lambda y EC2.*

2. Clase de Instancia (Instance configuration):

Confirme que la selección corresponda a **db.t3.micro** o **db.t4g.micro**. Ambas son elegibles para las 750 horas mensuales de la capa gratuita.

3. Almacenamiento (Storage - ¡Protección Anti-Cobros!):

- **Storage type:** General Purpose SSD (gp2 o gp3).
- **Allocated storage:** 20 GiB (Límite máximo gratuito).
- **Storage autoscaling (Autoescalado de almacenamiento):** **Desmarque** la casilla *Enable storage autoscaling*.
- *Justificación Crítica:* Si esta opción permanece activa y la base de datos alcanza su límite de 20 GB, AWS automáticamente comprará y asignará más almacenamiento, generando un cargo inmediato en la tarjeta de crédito. Para entornos de prueba, es preferible que el sistema rechace nuevas inserciones antes que generar facturación imprevista.

3.4.3 Conectividad y Aislamiento de Red

Esta es la configuración más sensible de la base de datos, ya que define su exposición al exterior.

1. Virtual private cloud (VPC):

En el menú desplegable, seleccione la red creada en la Fase 2: CloudAsset-vpc.

2. Public access (Acceso público - ¡Seguridad Crítica!):

Seleccione explícitamente **No**.

- *Justificación:* Esto evita que AWS asigne una dirección IP pública a la base de datos. El recurso quedará confinado en las subredes privadas, siendo matemáticamente imposible acceder a él directamente desde el internet público.

3. VPC security group (Firewall de red):

- Seleccione la opción **Choose existing** (Elegir existente).
- En la lista desplegable, busque y seleccione el grupo de seguridad que preparamos previamente: CloudAsset-RDS-SG.

- *Acción Preventiva:* Cierre (con la "X") cualquier grupo de seguridad predeterminado (default) que aparezca seleccionado junto al suyo, para evitar conflictos de reglas de red.

4. **Availability Zone (Zona de disponibilidad):** Seleccione *No preference* (Sin preferencia).

3.4.4 Inicialización de la Base de Datos

Para que nuestras aplicaciones puedan guardar datos inmediatamente sin tener que ejecutar comandos de inicialización manuales, le indicaremos a AWS que cree un esquema lógico interno durante el arranque del servidor.

1. Desplácese hacia la parte inferior y despliegue la sección **Additional configuration** (Configuración adicional).
2. **Initial database name (Nombre de la base de datos inicial):** Escriba exactamente la palabra inventario.
 - *Justificación:* Al proveer este nombre, el motor de RDS ejecutará internamente el comando SQL CREATE DATABASE inventario; antes de entregarnos el control del servidor. Esto previene errores de "Base de datos no encontrada" (Error 1049) en nuestras conexiones posteriores.
3. **Automated backups (Copias de seguridad automáticas):** Para este entorno de práctica, **desmarque** la opción *Enable automated backups*.
 - *Justificación de Costos:* Aunque la capa gratuita incluye 20 GB para backups, las instantáneas (snapshots) y copias automáticas pueden acumularse y consumir este espacio silenciosamente, generando cobros residuales. Desactivarlos elimina este riesgo durante el desarrollo.

3.4.5 Ejecución del Lanzamiento

1. Revise el panel derecho titulado **Estimated monthly costs** (Costos mensuales estimados). Debe indicar un mensaje confirmando que la instancia es elegible para la capa gratuita.
2. Haga clic en el botón naranja **Create database**.

(Nota Operativa: El aprovisionamiento de una instancia de RDS implica la asignación de hardware físico subyacente, instalación del motor MySQL y propagación de políticas de red. Este proceso toma habitualmente entre 5 y 10 minutos. El estado cambiará de "Creating" a "Available" cuando haya finalizado).

Validación de la Fase 4: El sistema ahora cuenta con un motor de bases de datos relacional altamente seguro. Está blindado contra el tráfico de internet, configurado estrictamente bajo los límites financieros de la capa gratuita, y pre-inicializado con el esquema inventario, listo para recibir inserciones de datos desde la función Lambda.

3.5 Fase 5: Automatización Serverless y Lógica de Negocio (AWS Lambda)

En esta etapa, implementaremos el componente de inteligencia de nuestra arquitectura: **AWS Lambda**. Este servicio representa el paradigma *Serverless* (sin servidor), permitiendo ejecutar lógica de backend únicamente cuando ocurre un evento específico, lo que garantiza una eficiencia de costos absoluta al no consumir recursos mientras el sistema está inactivo.

La función Lambda actuará como el "puente" integrador. Su flujo de trabajo consiste en interceptar la notificación de S3, extraer las dimensiones y el nombre del activo digital, y persistir estos metadatos en la tabla de la base de datos RDS.

A continuación, se detalla el procedimiento técnico para configurar la función, asegurando la conectividad de red privada y el enrutamiento correcto del código fuente para evitar errores de ejecución.

3.5.1 Definición de la Función y Entorno de Ejecución

1. **Acceso al Servicio:** En la consola de AWS, busque y seleccione **Lambda**.
2. **Creación Inicial:** Haga clic en **Create function** (Crear función).
3. **Configuración Básica:**
 - Seleccione **Author from scratch** (Crear desde cero).
 - **Function name:** Ingrese CloudAsset-Processor.
 - **Runtime (Tiempo de ejecución):** Seleccione **Node.js 20.x** (o la versión LTS más reciente).
 - **Architecture:** Mantenga la selección en **x86_64**.
4. **Asignación de Identidad (IAM):**
 - Despliegue la sección **Change default execution role**.
 - Seleccione **Use an existing role** (Usar un rol existente).
 - En el menú desplegable, elija el rol creado en la Fase 1: CloudAsset-Lambda-Role.
5. Haga clic en **Create function**.

3.5.2 Integración con la Red Privada (VPC)

Para que la función Lambda pueda "hablar" con la base de datos MySQL que reside en las subredes privadas, es imperativo otorgarle presencia dentro de la VPC.

1. En la interfaz de la función, navegue a la pestaña **Configuration** (Configuración) y seleccione la sección **VPC** en el menú izquierdo.

2. Haga clic en **Edit** (Editar).
3. **Configuración de Red:**
 - **VPC:** Seleccione CloudAsset-vpc.
 - **Subnets:** Seleccione exclusivamente las dos **subredes privadas** de su VPC.
 - **Security groups:** Seleccione el grupo de seguridad CloudAsset-Lambda-SG.
4. Haga clic en **Save**. *(Nota: Este proceso puede tomar hasta 2 minutos mientras AWS aprovisiona las interfaces de red elásticas).*

3.5.3 Configuración del Disparador (S3 Trigger)

1. En el diagrama visual de la función, haga clic en **+ Add trigger** (Agregar disparador).
2. Seleccione **S3** del menú desplegable.
3. **Bucket:** Busque y seleccione su bucket creado en la Fase 3 (cloudasset-inventario-...).
4. **Event types:** Mantenga **All object create events**.
5. Confirme la casilla de reconocimiento y haga clic en **Add**.

3.5.4 Ajustes Avanzados y Enrutamiento (Configuración Crítica)

Para garantizar un despliegue profesional y libre de errores de "módulo no encontrado", ajustaremos los parámetros de ejecución y la ruta del controlador (Handler).

1. **Ajuste de Tiempo de Espera (Timeout):**
 - En la pestaña **Configuration** -> **General configuration**, haga clic en **Edit**.
 - Cambie el **Timeout** de 3 segundos a **30 segundos**.
 - *Justificación:* Debido a que la función opera dentro de una VPC, el "Cold Start" (arranque en frío) y el establecimiento de la conexión TCP con RDS requieren un margen de tiempo superior al predeterminado.
2. **Enrutamiento del Controlador (Handler Mapping):**
 - En la misma pantalla de **Runtime settings**, localice el campo **Handler**.
 - Modifique el valor por defecto a: `lambda-code/index.handler`.
 - *Justificación Técnica:* Esta configuración le indica a AWS que el punto de entrada de la lógica se encuentra dentro de una subcarpeta denominada lambda-code, permitiendo mantener una estructura de proyecto organizada sin romper el flujo de ejecución.

3.5.5 Implementación del Código Fuente

El código debe empaquetarse en un archivo .zip que incluya la carpeta node_modules con la dependencia mysql2.

JavaScript

```
/* index.mjs - Lógica de Procesamiento de Metadatos */
import mysql from 'mysql2/promise';

export const handler = async (event) => {
  // Extracción de datos del evento S3
  const bucket = event.Records[0].s3.bucket.name;
  const key =
    decodeURIComponent(event.Records[0].s3.object.key.replace(/\+/g, ' '));
  const size = event.Records[0].s3.object.size;

  // Configuración de conexión (Utilizar variables de entorno en
  // producción)
  const dbConfig = {
    host: 'SU_ENDPOINT_DE_RDS',
    user: 'admin',
    password: 'SU_CONTRASEÑA',
    database: 'inventario'
  };

  let connection;
  try {
    connection = await mysql.createConnection(dbConfig);

    // Creación preventiva de la tabla estructurada
    await connection.execute(`
      CREATE TABLE IF NOT EXISTS archivos (
        id INT AUTO_INCREMENT PRIMARY KEY,
        nombre_archivo VARCHAR(255) NOT NULL,
        tamaño_bytes INT NOT NULL,
        fecha_subida DATETIME NOT NULL
      )
    `);

    // Inserción de metadatos analíticos
    await connection.execute(
      'INSERT INTO archivos (nombre_archivo, tamaño_bytes, fecha_subida) VALUES (?, ?, ?)',
      [key, size, new Date()]
    );

    return { statusCode: 200, body: "Registro exitoso" };
  } catch (error) {
    console.error("Error operativo:", error);
    throw error;
  } finally {
    if (connection) await connection.end();
  }
};
```

Validación de la Fase 5: La automatización ha sido establecida. El sistema ahora cuenta con la capacidad de reaccionar en tiempo real a la carga de datos, procesar la información de forma segura dentro de la red privada y registrar cada movimiento en la base de datos relacional, todo bajo una configuración de alto rendimiento y tolerancia a fallos.

3.6 Fase 6: Despliegue del Servidor Web y Visualización (Amazon EC2)

El último componente de la arquitectura es la capa de presentación. Utilizaremos **Amazon Elastic Compute Cloud (EC2)** para aprovisionar un servidor virtual que alojará una aplicación web desarrollada en Node.js.

Esta aplicación fungirá como un portal de lectura (interfaz de usuario) que consultará de forma segura la base de datos privada (RDS) y generará un reporte visual del inventario en tiempo real.

A continuación, se detalla el despliegue del servidor en la subred pública, la configuración mediante acceso seguro y la programación del código fuente, aplicando las correcciones necesarias para asegurar una conexión impecable.

3.6.1 Aprovisionamiento y Configuración de la Instancia

1. **Acceso al Servicio:** En la consola de AWS, busque y seleccione el servicio **EC2**.
2. **Inicio del Lanzamiento:** En el panel principal, haga clic en el botón naranja **Launch instance** (Lanzar instancia).
3. **Identificación y Sistema Operativo:**
 - **Name (Nombre):** Asigne el identificador CloudAsset-WebServer.
 - **Application and OS Images (AMI):** Seleccione **Amazon Linux**. Asegúrese de que la versión seleccionada sea la **Amazon Linux 2023 AMI** y que contenga la etiqueta *"Free tier eligible"*. Una **AMI (Amazon Machine Image)** es una plantilla maestra que contiene la configuración del sistema operativo.
4. **Capacidad de Cómputo e Ingreso:**
 - **Instance type:** Seleccione **t2.micro** o **t3.micro** (elegibles para la capa gratuita, proporcionando 750 horas mensuales).
 - **Key pair (Par de claves):** Haga clic en *Create new key pair*, nómbrelo cloudasset-key, seleccione formato .pem y guárdelo. Aunque utilizaremos un método más seguro para conectarnos, es una buena práctica de infraestructura generar esta clave criptográfica.
5. **Configuración de Red (Network settings - ¡Paso Crítico!):**
 - Haga clic en el botón **Edit** de esta sección.

- **VPC:** Seleccione su red CloudAsset-vpc.
- **Subnet:** Seleccione explícitamente una **Subred Pública** (Public Subnet). Esto ubicará el servidor en la zona accesible desde el exterior.
- **Auto-assign public IP:** Cambie esta opción a **Enable** (Habilitar). Sin una dirección IP pública, los usuarios no podrán navegar hacia la página web.
- **Firewall (Security groups):** Elija *Select existing security group* y seleccione CloudAsset-EC2-SG (el firewall configurado en la Fase 2 que permite el puerto 80).

6. Permisos Avanzados (IAM Integration):

- Despliegue la sección **Advanced details** al final de la página.
- Localice el campo **IAM instance profile** (Perfil de instancia IAM).
- Seleccione el rol creado en la Fase 1: CloudAsset-EC2-Role.
- *Justificación:* Esto otorga al servidor los permisos necesarios para ser administrado remotamente de forma segura sin exponer puertos sensibles.

7. Haga clic en **Launch instance**.

3.6.2 Conexión Segura mediante AWS Systems Manager

Para garantizar el más alto nivel de seguridad, accederemos al servidor utilizando **Session Manager**, una herramienta que establece un túnel cifrado mediante el rol de IAM, evadiendo la necesidad de utilizar el protocolo SSH tradicional.

1. En la consola de EC2, vaya a la lista de **Instances** (Instancias).
2. Espere a que el estado de la instancia CloudAsset-WebServer cambie a **Running** (En ejecución) y que las comprobaciones de estado indiquen *2/2 checks passed*.
3. Seleccione la instancia y haga clic en el botón superior **Connect** (Conectar).
4. Seleccione la pestaña **Session Manager** y presione el botón naranja **Connect**. Se abrirá una terminal de línea de comandos directamente en su navegador.
5. Escriba los siguientes comandos para cambiar al entorno estándar y al usuario administrativo correcto:

```
sudo su - ec2-user
```

3.6.3 Preparación del Entorno Operativo (Node.js)

Dentro de la terminal del servidor, prepararemos el entorno instalando las dependencias necesarias.

1. **Actualización del sistema operativo:**

```
sudo dnf update -y
```

2. Instalación del motor Node.js:

```
sudo dnf install -y nodejs
```

3. Creación del directorio de la aplicación e inicialización:

```
mkdir cloudasset-web
```

```
cd cloudasset-web
```

```
npm init -y
```

4. Instalación de librerías del proyecto:

Se requiere express para montar el servidor web y mysql2 para gestionar la conexión hacia RDS.

```
npm install express mysql2
```

3.6.4 Desarrollo del Servidor Web (server.js)

1. Abra el editor de texto integrado ejecutando:

```
nano server.js
```

2. Pegue cuidadosamente el siguiente código, asegurándose de reemplazar los valores de conexión de la base de datos con los datos reales de su instancia RDS.

JavaScript

```
const express = require('express');
const mysql = require('mysql2/promise');

const app = express();
const port = 80; // Puerto HTTP estándar habilitado en el Security Group

// Configuración de la base de datos (Sustituir con valores reales)
const dbConfig = {
  host: 'SU_ENDPOINT_DE_RDS.amazonaws.com', // Endpoint de la instancia
  RDS
  user: 'admin',
  password: 'SU_CONTRASEÑA',
  database: 'inventario'
};

app.get('/', async (req, res) => {
  let connection;
  try {
    // Establecer conexión con la base de datos privada
    connection = await mysql.createConnection(dbConfig);

    // Consultar el inventario, ordenando por la fecha de ingreso
    const [rows] = await connection.execute('SELECT * FROM archivos
ORDER BY fecha_subida DESC');
```

```

// Renderización de la Interfaz de Usuario (HTML)
let html = `
  <!DOCTYPE html>
  <html lang="es">
  <head>
    <meta charset="UTF-8">
    <title>CloudAsset Manager</title>
    <style>
      body { font-family: 'Segoe UI', Tahoma, Geneva,
Verdana, sans-serif; margin: 40px; background-color: #f4f4f9;}
      h1 { color: #ff9900; }
      .container { max-width: 1000px; margin: 0 auto;
background: white; padding: 20px; border-radius: 8px; box-shadow: 0 4px
6px rgba(0,0,0,0.1); }
      table { border-collapse: collapse; width: 100%;
margin-top: 20px; }
      th, td { padding: 12px; text-align: left; border-
bottom: 1px solid #ddd; }
      th { background-color: #232f3e; color: white; font-
weight: 500; }
      tr:hover { background-color: #f1f1f1; }
    </style>
  </head>
  <body>
    <div class="container">
      <h1>☁ CloudAsset Manager</h1>
      <p>Inventario de activos digitales sincronizado en
tiempo real desde Amazon RDS.</p>
      <table>
        <tr>
          <th>ID</th>
          <th>Nombre del Activo</th>
          <th>Tamaño (Bytes)</th>
          <th>Fecha de Registro</th>
        </tr>
      </table>
    </div>
  </body>
`;

if (rows.length === 0) {
  html += `<tr><td colspan="4" style="text-align: center;">El
inventario está vacío. Suba un archivo a Amazon S3.</td></tr>`;
} else {
  rows.forEach(row => {
    html += `
      <tr>
        <td>${row.id}</td>
        <td>${row.nombre_archivo}</td>
        <td>${row.tamaño_bytes.toLocaleString()}</td>
        <td>${new
Date(row.fecha_subida).toLocaleString('es-MX')}</td>
      </tr>
    `;
  });
}

```

```

    }

    html += `
        </table>
    </div>
</body>
</html>
`;

    res.send(html);
} catch (error) {
    console.error("Fallo de conexión a base de datos:", error);
    res.status(500).send(`<h2>Error interno del servidor</h2><p>No se
pudo conectar a la base de datos.</p>`);
} finally {
    if (connection) await connection.end();
}
});

// Inicialización del servicio de escucha
app.listen(port, () => {
    // Función de registro estandarizada
    console.log(`Servidor web operativo y escuchando en el puerto
${port}`);
});

```

3. Guarde el archivo presionando Ctrl + O, presione Enter para confirmar el nombre y presione Ctrl + X para salir del editor.

3.6.5 Ejecución y Verificación de Acceso

1. Para iniciar el servidor web con los privilegios necesarios para utilizar el puerto 80, ejecute:

sudo node server.js

La terminal deberá mostrar el mensaje: Servidor web operativo y escuchando en el puerto 80.

2. Acceso a la Interfaz:

- Regrese a la consola de EC2 en AWS y copie la **Dirección IPv4 pública** de su instancia (Ej. 3.142.12.241).
- Abra una nueva pestaña en su navegador web.
- **Regla Crítica de Red:** Para evitar errores de *Timeout* (tiempo de espera agotado) causados por protocolos predeterminados, asegúrese de teclear explícitamente el prefijo **http://** (sin la letra 's') seguido de su dirección IP: **http://SU_DIRECCION_IP_PUBLICA**

Validación de la Fase 6: Al acceder a la dirección indicada, el sistema presentará la interfaz de "CloudAsset Manager". Si la base de datos RDS ha registrado eventos previos provenientes de S3 y Lambda, la tabla mostrará de inmediato los metadatos de los activos digitales, comprobando el funcionamiento integral (End-to-End) de la arquitectura diseñada.

3.7 Fase 7: Observabilidad, Monitoreo y Alertas (Amazon CloudWatch)

El despliegue de una arquitectura en la nube no concluye con la confirmación de que sus componentes operan correctamente. En entornos empresariales de producción, es fundamental pasar de una postura reactiva (esperar a que un usuario reporte una falla) a una postura **proactiva** (detectar y resolver el problema antes de que afecte al usuario).

Para lograr esto, implementaremos **Amazon CloudWatch**, el servicio central de observabilidad de AWS. Este servicio recopila datos operativos y de rendimiento en forma de **Métricas** (variables numéricas a lo largo del tiempo, como el uso de CPU o cantidad de errores) y **Logs** (registros detallados de eventos).

En esta fase final, construiremos un panel de control visual para monitorear la salud de nuestra automatización *Serverless* y configuraremos un sistema de alarmas automatizado que nos notificará inmediatamente si nuestro servidor web experimenta una sobrecarga.

3.7.1 Creación del Tablero de Monitoreo (Dashboard)

Un **Dashboard** (Tablero) es un espacio de trabajo personalizable que unifica gráficos e información visual sobre los recursos de AWS, permitiendo a los ingenieros evaluar el estado del sistema de un solo vistazo.

Vamos a crear un gráfico que mida en tiempo real cuántas veces se ejecuta nuestra función Lambda y si alguna de esas ejecuciones resulta en un error.

1. **Acceso al Servicio:** En la barra de búsqueda de la consola de AWS, escriba **CloudWatch** y seleccione el servicio.
2. **Inicialización del Tablero:**
 - En el menú lateral izquierdo, expanda la sección de métricas y haga clic en **Dashboards** (Tableros).
 - Haga clic en el botón naranja **Create dashboard** (Crear tablero).
 - Asigne el nombre estructurado CloudAsset-Dashboard y presione **Create**.
3. **Configuración del Gráfico Visual (Widget):**
 - El sistema solicitará agregar un primer elemento visual. Seleccione **Line** (Gráfico de líneas) y posteriormente elija **Metrics** (Métricas) como la fuente de datos.
4. **Selección de Datos Telemétricos:**
 - En la lista de servicios proporcionada en la parte inferior, haga clic en **Lambda** y luego seleccione la agrupación **By Function Name** (Por nombre de función).

- Utilice la barra de búsqueda interna para localizar nuestra función: CloudAsset-Processor.
 - Marque las casillas correspondientes a dos métricas fundamentales: **Invocations** (Invocaciones: nos dirá el volumen de tráfico) y **Errors** (Errores: indicará fallos en el código o conectividad).
5. **Finalización:** Haga clic en el botón naranja **Create widget** en la esquina inferior derecha y, una vez en el tablero, presione **Save** (Guardar) en la esquina superior derecha.

3.7.2 Configuración del Sistema de Alertas Automáticas

El monitoreo visual es excelente, pero un administrador no puede observar una pantalla 24/7. Las **Alarmas** automatizan este proceso evaluando una métrica contra un umbral (threshold) definido y desencadenando acciones cuando se supera dicho límite.

Para proteger la integridad de nuestro portal web, crearemos una alarma que supervise el nivel de procesamiento del servidor EC2.

1. **Inicio de la Alarma:**

- En el menú izquierdo de CloudWatch, expanda la sección **Alarms** (Alarmas) y haga clic en **All alarms** (Todas las alarmas).
- Haga clic en el botón naranja **Create alarm** (Crear alarma).

2. **Definición de la Métrica a Evaluar:**

- Haga clic en **Select metric** (Seleccionar métrica).
- Navegue a través de la ruta: **EC2** -> **Per-Instance Metrics** (Métricas por instancia).
- Localice el identificador de su servidor (CloudAsset-WebServer) y marque la métrica denominada **CPUUtilization** (Utilización de CPU).
- Presione **Select metric** en la esquina inferior derecha.

3. **Configuración de Condiciones (Thresholds):**

- En la sección de condiciones, seleccione la opción **Greater/Equal** (Mayor o igual que).
- En el campo numérico (than...), ingrese el valor **80**.
- *Justificación Técnica:* Esto indica que si el servidor dedica el 80% de su capacidad de procesamiento de manera sostenida (lo que indica una sobrecarga inminente o un pico de tráfico anómalo), la alarma cambiará a estado "En Peligro" (In Alarm). Haga clic en **Next**.

4. **Configuración de Notificaciones y Amazon SNS:**

- Para enviar correos electrónicos, CloudWatch requiere integrarse con **Amazon Simple Notification Service (SNS)**, un servicio de mensajería masiva estructurado en "Temas" (Topics) y "Suscriptores".
- Seleccione la opción **Create new topic** (Crear un nuevo tema).
- **Topic name (Nombre del tema):** Escriba CloudAsset-Alerts.
- **Email endpoints (Puntos de enlace):** Ingrese la dirección de correo electrónico corporativa o personal del administrador del sistema.
- Haga clic en **Create topic**.
- **¡Requisito Crítico!** AWS no enviará alertas a un correo no verificado. En este instante, el sistema le enviará un correo electrónico. Debe abrir su bandeja de entrada y hacer clic en el enlace "**Confirm subscription**" (Confirmar suscripción). Sin este paso, las alertas fallarán silenciosamente.
- Haga clic en **Next**.

5. Nombramiento y Creación:

- Asigne a la alarma el nombre Alarma-CPU-Web e ingrese una breve descripción de su propósito.
- Haga clic en **Next**, revise el resumen arquitectónico de la alarma y presione **Create alarm**.

Validación de la Fase 7: Con la implementación de CloudWatch, el sistema *CloudAsset Manager* ha alcanzado la madurez operativa. Cuenta con una ventana visual hacia su flujo de procesamiento *Serverless* y un mecanismo de alerta temprana que salvaguarda la disponibilidad de la infraestructura pública, enviando reportes críticos directamente a los administradores correspondientes.

Capítulo 4: Pruebas de Integración y Resolución de Problemas

Para garantizar que la arquitectura implementada opera con la fiabilidad requerida en un entorno de producción, es imperativo ejecutar un protocolo de pruebas. Las **Pruebas de Extremo a Extremo (End-to-End Testing)** validan que el flujo de datos transite correctamente por todos los subsistemas, desde el detonador inicial hasta la capa de presentación.

4.1 Protocolo de Pruebas de Funcionamiento

Ejecute los siguientes pasos en orden secuencial para certificar la salud de la infraestructura:

Prueba 1: Inserción y Disparo de Eventos (S3 a Lambda)

1. Navegue a la consola de **Amazon S3** y acceda a su bucket (cloudasset-inventario-...).

2. Haga clic en **Cargar** (Upload) y suba un archivo de prueba (ej. una imagen o un documento PDF).
3. **Criterio de Éxito:** El archivo debe aparecer listado en el bucket. Internamente, S3 debe enviar un evento asíncrono a Lambda de forma instantánea.

Prueba 2: Trazabilidad y Lógica (AWS CloudWatch)

1. Navegue a **CloudWatch** -> **Grupos de registros** (Log groups) y seleccione el registro correspondiente a `/aws/lambda/CloudAsset-Processor`.
2. Abra el flujo de registro (Log stream) más reciente.
3. **Criterio de Éxito:** Debe existir un registro impreso que confirme el inicio y fin de la ejecución sin mensajes de nivel ERROR.

Prueba 3: Visualización en Tiempo Real (Integración Completa)

1. Abra una nueva pestaña en su navegador web.
2. Ingrese la dirección IP pública de su servidor EC2 utilizando estrictamente el protocolo HTTP (ej. `http://SU_DIRECCION_IP`).
3. **Criterio de Éxito:** La interfaz web debe cargar correctamente y la tabla debe mostrar una nueva fila con el ID, nombre, tamaño y fecha del archivo exacto que cargó en la Prueba 1.

4.2 Resolución de Errores Comunes (Troubleshooting)

Durante el despliegue de infraestructuras complejas, es habitual enfrentar desafíos de configuración. A continuación, se documentan los errores más frecuentes en este tipo de arquitectura y sus soluciones definitivas:

- **Error: El navegador muestra ERR_CONNECTION_TIMED_OUT al acceder al servidor web.**
 - **Causa Técnica:** Los navegadores modernos fuerzan la navegación segura intentando conectar por el puerto 443 (`https://`). Debido a que nuestro servidor Node.js escucha tráfico en el puerto estándar 80 (`http://`), el firewall de AWS bloquea la conexión por el puerto 443, provocando que el tiempo de espera se agote.
 - **Solución:** Escriba explícitamente el prefijo `http://` antes de la dirección IP en la barra de direcciones del navegador.
- **Error: Runtime.ImportModuleError: Error: Cannot find module 'index' en los logs de Lambda.**
 - **Causa Técnica:** Al comprimir el código en un archivo .zip, el archivo principal (`index.mjs` o `index.js`) quedó encapsulado dentro de una subcarpeta (por

ejemplo, lambda-code), pero el entorno de AWS Lambda está configurado para buscarlo en la raíz principal.

- **Solución:** Navegue a la configuración de "Tiempo de ejecución" (Runtime settings) de la función Lambda y actualice el campo **Controlador (Handler)** para que apunte a la ruta exacta, por ejemplo: lambda-code/index.handler.
- **Error en la interfaz web: Table 'inventario.archivos' doesn't exist.**
 - **Causa Técnica:** El servidor web logró conectarse a la base de datos RDS, pero la tabla no ha sido creada. En nuestra arquitectura, la creación de la tabla está delegada a la función Lambda (para evitar operaciones manuales de base de datos).
 - **Solución:** Simplemente suba su primer archivo al bucket de S3. Esto activará a Lambda, quien ejecutará la sentencia CREATE TABLE IF NOT EXISTS antes de insertar el primer registro.
- **Error: Falla al intentar conectar al servidor EC2 mediante "EC2 Instance Connect".**
 - **Causa Técnica:** El Grupo de Seguridad no tiene habilitada una regla de entrada para el protocolo SSH (Puerto 22).
 - **Solución Segura:** En lugar de abrir el puerto 22 al público, utilice la pestaña **Session Manager** para conectarse al servidor. Esta herramienta utiliza el Rol de IAM asignado a la instancia para establecer un túnel seguro directamente desde el navegador, cumpliendo con los más altos estándares de seguridad corporativa.

Conclusión del Manual

La construcción del **CloudAsset Manager** demuestra la capacidad de diseñar, desplegar y asegurar una infraestructura en la nube bajo el paradigma de *Arquitectura Orientada a Eventos*.

Al completar este proyecto, se ha transitado por las disciplinas fundamentales del *Cloud Computing*: Gestión de Identidades (IAM), Redes Virtuales Aisladas (VPC), Almacenamiento de Objetos (S3), Bases de Datos Relacionales (RDS), Computación Sin Servidor (Lambda), Servidores Virtuales (EC2) y Observabilidad (CloudWatch). Todo ello, logrando mantener la optimización financiera al operar íntegramente dentro de los límites de la Capa Gratuita de AWS.

El sistema resultante no es un simple prototipo, sino una base sólida y escalable que replica las prácticas de ingeniería de software utilizadas por las organizaciones tecnológicas líderes a nivel global.

Referencias y Bibliografía Técnica

- **Amazon Web Services.** (2026). *AWS Identity and Access Management (IAM) User Guide*. Recuperado de la documentación oficial de AWS.
- **Amazon Web Services.** (2026). *Amazon Virtual Private Cloud User Guide*. Recuperado de la documentación oficial de AWS.
- **Amazon Web Services.** (2026). *AWS Lambda Developer Guide: Event-driven invocations*. Recuperado de la documentación oficial de AWS.
- **Amazon Web Services.** (2026). *Amazon Relational Database Service (RDS) User Guide*. Recuperado de la documentación oficial de AWS.
- **Amazon Web Services.** (2026). *Amazon EC2 User Guide for Linux Instances*. Recuperado de la documentación oficial de AWS.